به نام خدا

استاد: خانم رضایی | گزارش کار آزمایشگاه سیستم دیجیتال ۲ (جلسه دوم) | علی سپهرنیا (۹۶۱۲۱۰۳۸)

۱- اتصال کی پد به میکرو و نمایش اعداد فشرده شده روی سون سگمنت

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم. در تب Ports پورت B را انتخاب میکنیم و همه ی بیت های آن را روی حالت Out یا همان ۱ میگذاریم تا بتوانیم روی آن بنویسیم و سون سگمنت را به آن وصل کنیم. همچنین در PORTD بیت های 2…0 را ورودی همراه با مقاومت بالاکش (P) و ۴ بیت پر ارزش را خروجی تعیین میکنیم. بقیه تنظیمات را تغییر نمی دهیم. مطمئن میشویم تیک گزینه Program > Generate Code For Disabled Peripherals برداشته شده است. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. در تابع main آرایه sevenseg را تعریف میکنیم و کد هگز اعداد 0 تا 9 برای سون سگمنت کاتد مشترک را در آن ذخیره میکنیم. متغیر key را نیز برای ذخیره عدد فشرده شده تعریف میکنیم. تابع ()ReadKeypad را برای خواندن عدد فشرده شده در کیپد به این صورت تعریف می کنیم که به سطر های کیپد (PORTD4…7) عدد ۱ منتسب می شود و سپس ۴ بار و در هر بار یک سطر را ۰ میکنیم و در این حین تک تک ستون های کیپد (PIND0…2) را میخوانیم و اگر هر کدام ۰ شد یعنی کلید متناظر با آن ستون و سطری که ۰ کردیم فشرده شده پس عدد آن را بر میگردانیم. در پایان این تابع اگر هیچکدام از کلیدها فشرده نشده بود عدد 1- را بر میگردانیم. در حلقه بینهایت یک حلقه do-while تکرار می شود که هر بار کیپد را میخواند و مقدار آن را در key ذخیره میکند و تا زمانی که کلیدی فشرده نشده باشد یعنی مقدار برگشتی تابع 1- باشد این کار را تکرار میکند. در پایان کد هگز معادل عدد key در پورت B نوشته می شود. از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که با هر بار اجرای حلقه بینهایت، حلقه do-while تکرار میشود. این حلقه مداوم کیپد را اسکن میکند تا یک مقدار معتبر برگردانده شود. سپس مقدار مذکور را روی پورت B می نویسد. بدین ترتیب میکروکنترلر کد هگز معادل مقدار آخرین کلید فشرده شده را در پورت B می نویسد. با وصل کردن سون سگمنت به این پورت میتوانیم نتیجه را مشاهده کنیم.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم.کامپوننت های سون سگمنت کاتد مشترک، کیپد ۴*۳ و میکروکنترلر ATMEGA16 را با جستجو در Device ها پیدا میکنیم و به پروژه اضافه میکنیم. میکروکنترلر،سون سگمنت و کیپد را به شماتیک اضافه میکنیم. پایه های دیتای سون سگمنت را به پورت B وصل میکنیم. کاتد مشترک را به زمین متصل میکنیم. ستون های ۱ تا ۳ کیپد را به بیت های 2…0 و سطرهای A تا D را به بیت های 7…4 از پورت D وصل میکنیم. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که سون سگمنت مقدار هر کلید فشرده شده را نمایش میدهد.

۲- برنامه ای بنویسید که عدد ۱۲۳۴ را روی سون سگمنت چهارتایی نمایش دهد

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم. در تب Ports پورت B را انتخاب میکنیم و ۴ بیت کم ارزش آن را روی حالت خروجی میگذاریم تا کاتد مشترک ارقام سون سگمنت را با آن کنترل کنیم. همچنین همه ی بیت های پورت C را روی حالت Out یا همان ۱ میگذاریم تا بتوانیم روی آن بنویسیم و سون سگمنت را به آن وصل کنیم. بقیه تنظیمات را تغییر نمیدهیم. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. در حلقه بی نهایت ابتدا روی ۴ بیت کم ارزش پورت B مقدار 1110 را مینویسیم تا رقم یکان سون سگمنت روشن شود. سپس کد هگز معادل رقم 4 را روی پورت C می نویسیم و در انتها یک delay به اندازه 1ms قرار میدهیم تا در عین حال که ارقام مختلف سون سگمنت را روشن و خاموش میکنیم، چشم بیننده متوجه این تغییر نشود. همین قطعه کد را برای برای ارقام دیگر تکرار میکنیم به طوری که برای ارقام دهگان تا هزارگان سون سگمنت، ۴ بیت کم ارزش پورت B را به ترتیب برابر 1101، 1011 و 0111 قرار می دهیم و کد هگز معادل ارقام 3، 2 و 1 را روی پورت C می نویسیم. از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که با هر بار اجرای حلقه بینهایت، در فاصله زمانی بسیار کوتاهی (1ms) ارقام سون سگمنت یک به یک فعال و غیرفعال می شوند و به این صورت اعداد 4 تا 1 را روی رقم ها یکان تا هزارگان سون سگمنت نمایش می دهد. از آنجایی که چشم انسان قادر به تشخیص تغییرات در این زمان کوتاه نیست و این حلقه تکرار می شود، همواره عدد 1234 روی سون سگمنت دیده می شود.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم. کامپوننت های سون سگمنت ۴ رقمی کاتد مشترک و میکروکنترلر ATMEGA16 را با جستجو در Device ها پیدا میکنیم و به پروژه اضافه میکنیم. میکروکنترلر و سون سگمنت را به شماتیک اضافه میکنیم. پایه های دیتای سون سگمنت را به پورت C و کاتد های مشترک ارقام یکان تا هزارگان را به ترتیب به پایه های 0…3 پورت B وصل میکنیم. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که عدد 1234 روی سون سگمنت مشاهده می شود.

پیوست ها:

آزمایش ۱)

G:\Personal\Uni\Micro Lab\Micro\Az2-1\Az2-1.c

Notes | Az2-1.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session2-Experiment1
Version :
Date    : 10/7/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>
#include <delay.h>

int ReadKeypad();

// Declare your global variables here

void main(void)
{
// Declare your local variables here
int sevenseg[] = {0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f};
int key;
```

G:\Personal\Uni\Micro Lab\Micro\Az2-1\Az2-1.c

Notes | Az2-1.c

```
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
DDRB=(1<<DDB7) | (1<<DDB6) | (1<<DDB5) | (1<<DDB4) | (1<<DDB3) | (1<<DDB2) | (1<<DDB1) | (1<<DDB0);
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=(1<<DDD7) | (1<<DDD6) | (1<<DDD5) | (1<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=T Bit2=P Bit1=P Bit0=P
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (1<<PORTD2) | (1<<PORTD1) | (1<<PORTD0);

while (1)
      {
      // Place your code here
        do
            key = ReadKeypad();
        while(key<0);

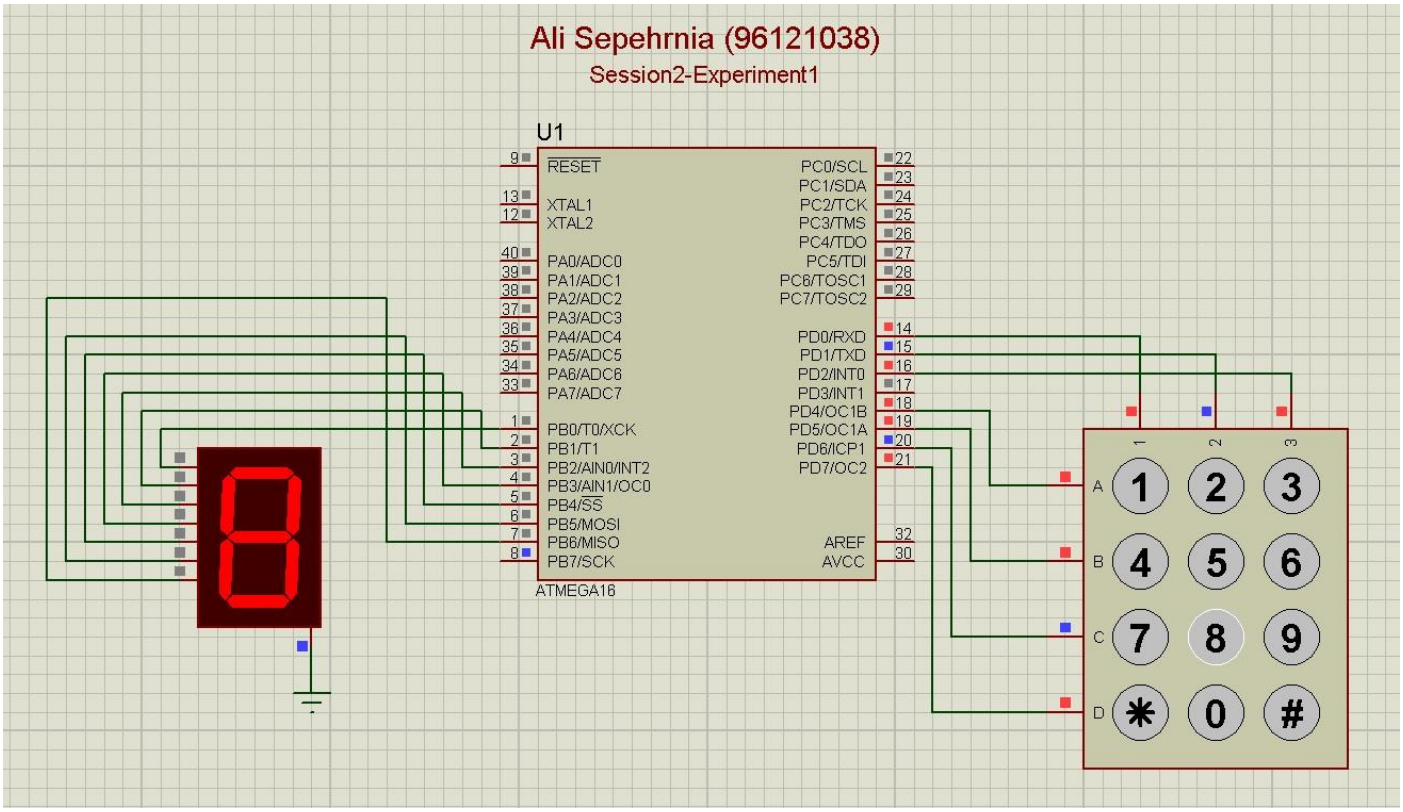
        PORTB = sevenseg[key];
        delay_ms(100);
      }
}
```

Line: 38

G:\Personal\Uni\Micro Lab\Micro\Az2-1\Az2-1.c

Notes | Az2-1.c

```
73
74  int ReadKeypad()
75  {
76      PORTD.4 = 1;
77      PORTD.5 = 1;
78      PORTD.6 = 1;
79      PORTD.7 = 1;
80
81      PORTD.4 = 0;
82      if(PIND.0 == 0) return 1;
83      else if(PIND.1 == 0) return 2;
84      else if(PIND.2 == 0) return 3;
85      PORTD.4 = 1;
86
87      PORTD.5 = 0;
88      if(PIND.0 == 0) return 4;
89      else if(PIND.1 == 0) return 5;
90      else if(PIND.2 == 0) return 6;
91      PORTD.5 = 1;
92
93      PORTD.6 = 0;
94      if(PIND.0 == 0) return 7;
95      else if(PIND.1 == 0) return 8;
96      else if(PIND.2 == 0) return 9;
97      PORTD.6 = 1;
98
99      PORTD.7 = 0;
100     if(PIND.1 == 0) return 0;
101     PORTD.7 = 1;
102
103     return -1;
104 }
```



آزمایش ۲)

G:\Personal\Uni\Micro Lab\Micro\Az2-2\Az2-2.c

Notes | Az2-2.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session2-Experiment2
Version :
Date    : 10/7/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>
#include <delay.h>

// Declare your global variables here

void main(void)
{
// Declare your local variables here

// Input/Output Ports initialization
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
```

G:\Personal\Uni\Micro Lab\Micro\Az2-2\Az2-2.c

Notes | Az2-2.c

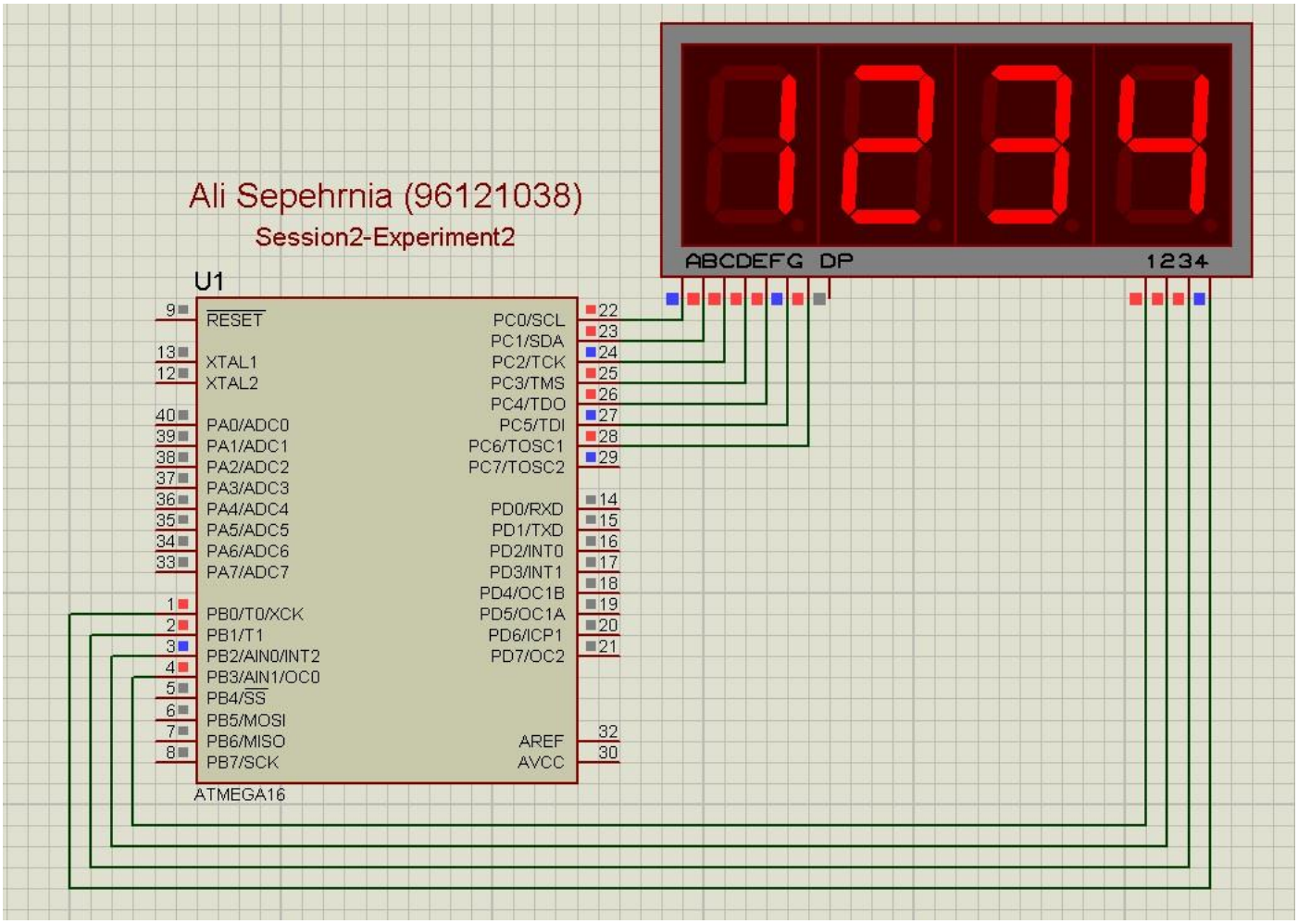
```
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=Out Bit2=Out Bit1=Out Bit0=Out
DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (1<<DDB3) | (1<<DDB2) | (1<<DDB1) | (1<<DDB0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
DDRC=(1<<DDC7) | (1<<DDC6) | (1<<DDC5) | (1<<DDC4) | (1<<DDC3) | (1<<DDC2) | (1<<DDC1) | (1<<DDC0);
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

while (1)
      {
        PORTB = 0b00001110;
        PORTC = 0x66;
        delay_ms(1);
        PORTB = 0b00001101;
        PORTC = 0x4f;
        delay_ms(1);
        PORTB = 0b00001011;
        PORTC = 0x5b;
        delay_ms(1);
        PORTB = 0b00000111;
        PORTC = 0x06;
        delay_ms(1);
```


Ali Sepehrnia (96121038)
Session2-Experiment2
1234
ABCDEFG DP
1234
U1
RESET
XTAL1
XTAL2
PA0/ADC0
PA1/ADC1
PA2/ADC2
PA3/ADC3
PA4/ADC4
PA5/ADC5
PA6/ADC6
PA7/ADC7
PB0/T0/XCK
PB1/T1
PB2/AIN0/INT2
PB3/AIN1/OC0
PB4/SS
PB5/MOSI
PB6/MISO
PB7/SCK
ATMEGA16
PC0/SCL
PC1/SDA
PC2/TCK
PC3/TMS
PC4/TDO
PC5/TDI
PC6/TOSC1
PC7/TOSC2
PD0/RXD
PD1/TXD
PD2/INT0
PD3/INT1
PD4/OC1B
PD5/OC1A
PD6/ICP1
PD7/OC2
AREF
AVCC